

Liquidity Delivery PR #80 MoonPay Integration Audit Report

Table of Contents

1. Executive Summary

4

About Liquidity Delivery

4

Audit Summary

4

Overall Security Posture

4

Launch Recommendations

5

Audit Scope

6

2. Assumptions and Considerations

7

Audit Assumptions

7

Centralization Risks

7

Trust Assumptions

7

3. Severity Definitions

8

Impact

8

Likelihood

8

Severity Classification Matrix

8

4. Findings

10

M01: u64 conversion in calculate_fill rejects fill reports for orders with amount_out > u64::MAX

10

L01: Front-run of initialize grants the first caller admin and the portal_authority escrow-release gate

14

5. Enhancement Opportunities

16

E01: ReportOrderFill::validate does not check the reported token_in or reject a zero recipient

16

E02: set_portal_authority rotates the escrow-release authority without staging or an event

18

E03: Writing Nonce.bump on initialization will complete the account's state

20

E04: origin_recipient zero-check is present on FillForeignOrder but absent on FillNativeOrder

21

E05: Orders cannot be filled until the client's created_at guess expires

23

About Us25

About Adevar Labs25

 Audit Methodology25

 Confidentiality Notice27

 Legal Disclaimer27

1. Executive Summary

About Liquidity Delivery

M0 provides a shared liquidity delivery system that ensures smooth movement between M0-powered stablecoins and major existing stablecoins (USDC, USDT, etc.). This lets anyone using an M0 extension easily convert into and out of it with other stablecoins across multiple chains. Core components include on-chain order contracts (Solana + EVM), a Portal bridge for reporting fills/cancels back to the origin, and solver/relayer flows to execute fills and propagate reports.

This audit covers PR #80 and its related changes, whose goal is to port the EVM v1.1 funder/sender split to SVM when opening an order: an explicit sender now holds cancel rights and receives the canceled funds, while tokens still come from the funding account.

Audit Summary

The table below summarizes identified vulnerabilities by severity and remediation status.

Severity	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	1	1	0
Low	1	0	1

Enhancement Opportunities: 5

Overall Security Posture

The reviewed change set is small, well structured, and consistent with the EVM implementation it ports. The risks concentrate at the boundaries: integer widths and validation coverage diverge between the cross-chain paths, and initialization and **portal_authority** rotation lack the safeguards the admin role already has. The Medium issue has been fixed as recommended, and the remaining items were acknowledged with operational justifications.

After Fix Review Summary

The Medium severity issue is fixed according to the recommendation.

The Low severity **initialize** front-run was acknowledged. The developers informed us that the program is already deployed and initialized, and that for a new deployment they would verify the config after **initialize** and close and redeploy the program if needed. The residual risk is limited to future deployments and depends on that procedure being followed.

Finally, the 5 enhancement opportunities were acknowledged. These items harden consistency between code paths rather than address exploitable flaws, and the team chose to defer them, judging the current safeguards and solver behavior sufficient for this release.

Before Fix Review Summary

The review identified 1 Medium and 1 Low severity issues, and no Critical or High severity issues. The Medium issue is an unnecessary u128 to u64 narrowing in **calculate_fill**: orders whose **amount_out** exceeds **u64::MAX**, which is roughly 18.4 units of an 18-decimal token, cause the fill report to revert permanently, leaving the escrowed **token_in** locked while the order stays in **Created** state on Solana, while it is considered **Completed** on the destination chain. The Low issue is a deployment-window front-run: **initialize** binds no known authority, so the first caller after program deployment becomes **admin** and chooses **portal_authority**, the only key able to release cross-chain escrow, and recovery requires redeploying at a new program ID.

The 5 enhancement opportunities share a theme of asymmetry: a check or convention applied on one path but not its counterpart. **FillNativeOrder::validate** omits the zero **origin_recipient** check that **FillForeignOrder::validate** already performs. **ReportOrderFill::validate** settles fill reports without the **token_in** match and zero-recipient checks its cancel counterpart makes. **set_portal_authority** rotates the escrow-release authority in a single unstaged instruction with no event, unlike the staged **admin** handover. The **Nonce** PDA never persists its **bump** even though every other PDA in the program does. Finally, the forward-looking **created_at** window forces clients to guess a future timestamp, which is the opposite of the intent stated in the code comment and delays when an order becomes fillable.

Launch Recommendations

After Fix Review

- The mandatory recommendation was implemented: **calculate_fill** returns **u128**, narrowing happens only at the SPL transfers, and the partial-fill product uses a 256-bit **muldiv_u128**.
- The other recommended items were not implemented as part of this review.

Before Fix Review

Mandatory before launch

- Address the Medium severity issue: return **u128** from **calculate_fill**, narrow to **u64** only at the SPL transfer call sites, and widen the partial-fill intermediate product so the **total_amount_in * amount_out_to_fill** multiplication cannot overflow once the narrowing is removed.

Recommended

- Close the **initialize** front-run window: require a deployer co-signer on **initialize**, or deploy and initialize atomically, so the first caller cannot become **admin** and take the **portal_authority** escrow-release gate.
- Mirror the **token_in** match and zero **origin_recipient** checks from **ReportOrderCancel::validate** into **ReportOrderFill::validate** so mismatched or unclaimable fill reports are rejected instead of settled.
- Stage the **portal_authority** handover the way **admin** is staged, and emit an event on rotation so indexers can observe it.
- Mirror the **origin_recipient** zero-check from **FillForeignOrder::validate** into **FillNativeOrder::validate** so both fill paths reject the same invalid input.
- Persist **ctx.bumps.sender_nonce_account** into **Nonce.bump** on first use, restoring the PDA convention the rest of the program follows.
- Reverse the **created_at** validation window to accept timestamps in the recent past, and update **open-order.ts** to send the real creation time with no buffer.

Post-launch monitoring

- Alert on **InvalidFillAmount** reverts in **report_order_fill**, **FillNativeOrder**, and **FillForeignOrder**. Any occurrence means a fill report cannot be settled and escrowed funds are stuck.
- Monitor cancel transactions to confirm that canceled funds are returned to the sender account rather than the funder.

Audit Scope

- **Repository:** <https://github.com/m0-platform/liquidity-delivery>
- **Before Fix Review Commit Hash:** **6bec28cbceef75f46960b14a24be44a0221c1a0**
- **After Fix Review Commit Hash:** **f92916a6d207f81edfded307d19feb888009e2a9**
- **Files/Modules in Scope:**
 - PR #80: changed ***.rs** files in **svm/programs**

2. Assumptions and Considerations

Audit Assumptions

1. Portal authority honesty and delivery: The portal authority behaves correctly, forwards **ReportFill/ReportCancel** messages from destination to origin, and does not replay them (reports are not idempotent by default).
2. Recipient smart contracts (cross-chain): If a cross-chain recipient is a smart contract, it can cancel before the deadline when needed. Otherwise, an infinite deadline could strand funds.
3. Already known issue: Anyone can increment any sender's **Nonce.value** by opening a self-funded order, causing the legitimate sender's order to fail. The developers prefer to keep SVM/EVM parity, as the impact and likelihood of such an event are low.

Centralization Risks

- Admin/pauser roles can pause/unpause and update destinations.
- Portal authority controls cross-chain report submission. Misuse can block or distort fills/cancels.
- Bridge adapter choice and Portal dependency centralize cross-chain delivery.

Trust Assumptions

- Portal bridge and relayers deliver messages faithfully.
- Off-chain solvers/relayers follow expected workflows (e.g., include correct accounts, send reports).

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions convey our team's understanding of both the potential consequences of a vulnerability and how likely it is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements, typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue, particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easily an attacker can discover and exploit a vulnerability, as well as how economically attractive the exploit is.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risk.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but the financial risk remains manageable.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically infeasible or requires highly specialized conditions. They may require extraordinary effort or entail significant financial risk for an attacker, with a high chance of failure and minimal potential return.

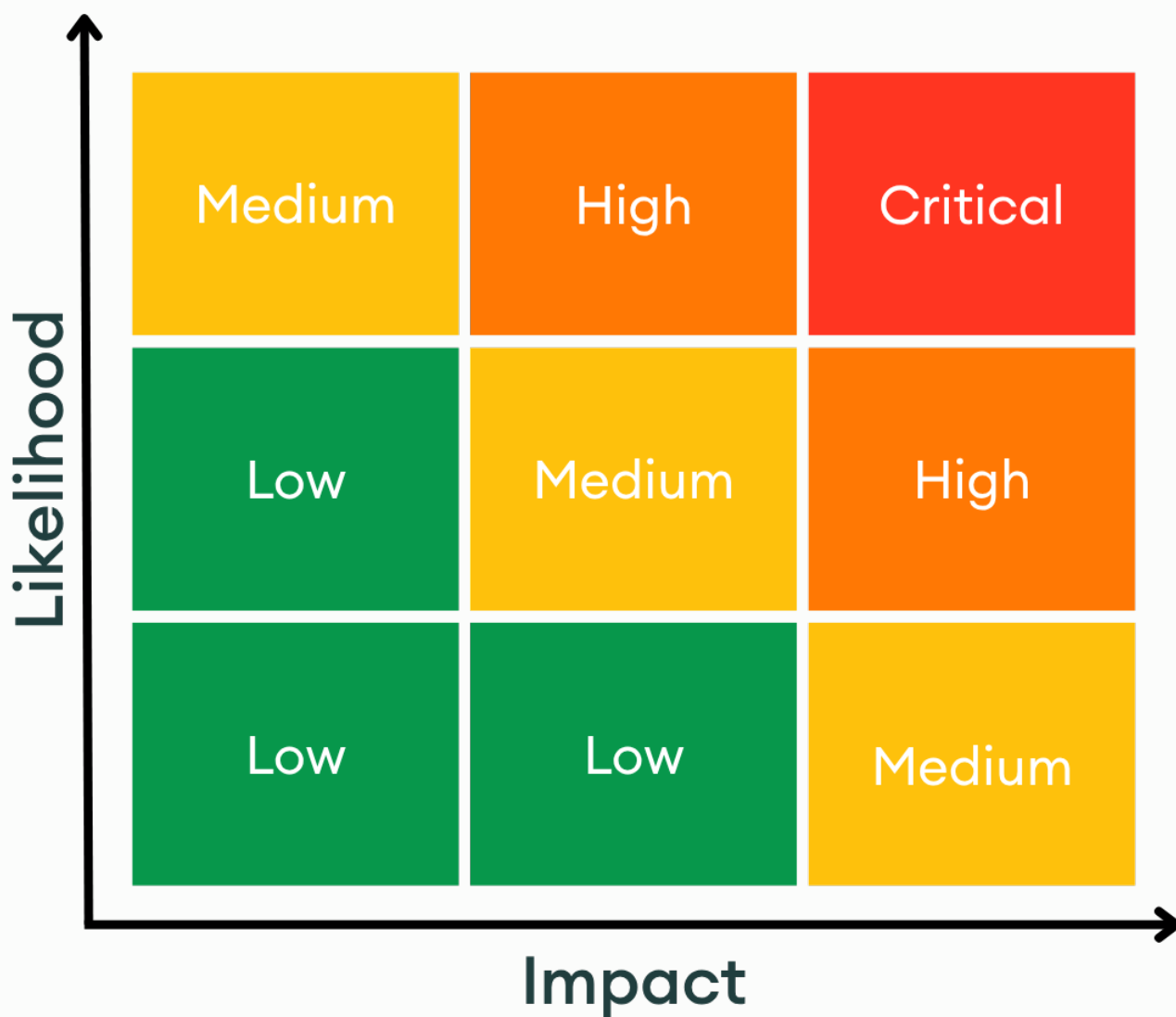
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact with high likelihood (e.g., a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Medium impact and likelihood, or high impact/likelihood with low likelihood/impact

- **Low:** Low impact and likelihood, or medium impact/likelihood with low likelihood/impact

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

M01: u64 conversion in `calculate_fill` rejects fill reports for orders with `amount_out > u64::MAX`



LOCATION

[liquidity-delivery/6bec28cb/svm/programs/order_book/src/instructions/fill.rs:L709-L713](#)

RELEVANT SNIPPET

```
> _fill.rs RUST
707:     };
708:
709:     Ok((
710:         full_fill_,
711:         amount_in_to_release_.try_into().map_err(|_| OrderBookError::InvalidFillAmount)
712:     )?,
713:     amount_out_to_fill_.try_into().map_err(|_| OrderBookError::InvalidFillAmount)?
714: ))
```

OTHER LOCATIONS

- [liquidity-delivery/6bec28cb/svm/programs/order_book/src/instructions/fill.rs:L620-L626](#)
- [liquidity-delivery/6bec28cb/svm/programs/order_book/src/instructions/fill.rs:L411](#)

DESCRIPTION

When a cross-chain order is opened on Solana, the `token_in` is escrowed there and the fill happens on the destination chain. The `ReportFill` then travels back over Portal, and `report_order_fill` is the origin-chain leg that releases the escrowed `token_in` to the solver.

That handler calls `calculate_fill` to obtain the `full_fill` boolean and discards both amounts. But `calculate_fill` returns `(bool, u64, u64)` and converts both values with `try_into()` before returning. This makes sense when the values are used for SPL token transfers, which accept `u64` amounts.

```

>_ fill.rs RUST
707:     };
708:
709:     Ok((
710:         full_fill_,
711:         amount_in_to_release_.try_into().map_err(|_| OrderBookError::InvalidFillAmount
712:     )?),
713:         amount_out_to_fill_.try_into().map_err(|_| OrderBookError::InvalidFillAmount)?
714:     ))
715: }

```

However, it is wrong here as **amount_out** is expressed in the destination chain's token: **ReportFill** declares it **u128**, the order stores it as **u128**, and **open_order** only checks that it is not equal to zero when creating the order, consistent with the EVM side also carrying it as **uint128**.

Once **order.amount_out** exceeds **u64::MAX**, the conversion in **calculate_fill** fails and the instruction reverts. Because the instructions read the values from the order account data, every retry of the report will fail.

u64::MAX is about 18.4 units of an 18-decimal token, so an order paying out 1000 DAI (**1e21**) on Ethereum would trigger the revert.

This causes the escrowed **token_in** to be locked and inaccessible to the solver, as the order stays in **Created** state on Solana, while it is considered **Completed** on the destination chain.

FillForeignOrder is affected by the same down conversion, but with a lesser impact.

amount_in_to_release is denominated on the origin chain and is only included in a **u128** fill report. If a fill releases more than **u64::MAX** units from the origin chain, the instruction reverts. Here, the solver does not lose tokens because the transaction is atomic, but the order cannot be filled.

PROOF OF CONCEPT

1. Alice opens on Solana: **amount_in = 1_000e6**, **dest_chain_id = 1**, **token_out = DAI**, **amount_out = 1e21**.
2. A solver fully fills on Ethereum, paying 1000 DAI (**1e21** base units), and a **ReportFill** with **amountOutFilled = 1e21** is dispatched.
3. On Solana, **calculate_fill(1_000e6, 1e21, 0, 0, 1e21)** sets **full_fill_ = true** and **amount_out_to_fill_ = 1e21**.
4. **1e21.try_into::<u64>()** fails, and the transaction reverts.

RECOMMENDATION

Return **u128** from **calculate_fill**:

DIFF

```

-) -> Result<(bool, u64, u64)> {
+) -> Result<(bool, u128, u128)> {
...
    Ok((
        full_fill_,
        amount_in_to_release_.try_into().map_err(|_| OrderBookError::InvalidFillAmount)?,
        amount_out_to_fill_.try_into().map_err(|_| OrderBookError::InvalidFillAmount)?
    +    amount_in_to_release_,
    +    amount_out_to_fill_
    ))

```

And convert down only at the **FillNativeOrder::handler** where the values are SPL amounts:

DIFF

```

-    order.amount_in_released += amount_in_to_release as u128;
-    order.amount_out_filled += amount_out_to_fill as u128;
+    order.amount_in_released += amount_in_to_release;
+    order.amount_out_filled += amount_out_to_fill;

    transfer_exact_tokens(
        &ctx.accounts.solver_token_out_account,
        &mut ctx.accounts.recipient_token_out_ata,
        amount_out_to_fill,
    +    amount_out_to_fill.try_into().map_err(|_| OrderBookError::InvalidFillAmount)?,
    ...

    transfer_exact_tokens_from_program(
        &ctx.accounts.order_token_in_ata,
        &mut ctx.accounts.solver_token_in_account,
        amount_in_to_release,
    +    amount_in_to_release.try_into().map_err(|_| OrderBookError::InvalidFillAmount)?,

```

For **FillForeignOrder::handler**, only **amount_out_to_fill** is an SPL amount, so only this one should be converted down:

DIFF

```

transfer_exact_tokens(
    &ctx.accounts.solver_token_out_account,
    &mut ctx.accounts.recipient_token_out_ata,
    -    amount_out_to_fill,
    +    amount_out_to_fill.try_into().map_err(|_| OrderBookError::InvalidFillAmount)?,

```

Finally, widen the intermediate product in the partial-fill branch.

Once the **u64** narrowing is removed, **total_amount_in_ * amount_out_to_fill_** can potentially exceed **u128::MAX** and revert with **MathOverflow**, even though the quotient always fits:

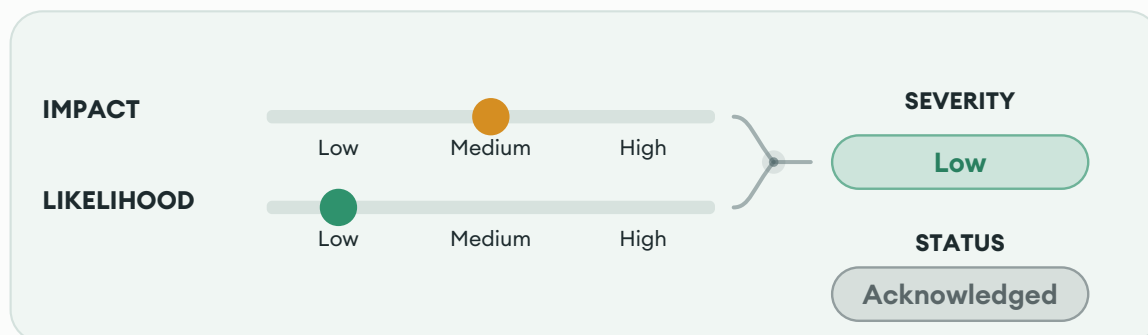
DIFF

```
let amount_in_to_release_ = if full_fill_ {  
    total_amount_in_.checked_sub(amount_in_released_).ok_or(OrderBookError::MathUnderflow  
)? // remaining amount  
} else {  
-    total_amount_in_.checked_mul(amount_out_to_fill_).ok_or(OrderBookError::MathOverflow)  
?  
-    .checked_div(total_amount_out_).ok_or(OrderBookError::MathUnderflow)?  
+    muldiv_u128(total_amount_in_, amount_out_to_fill_, total_amount_out_)? // 256-bit  
    intermediate  
};
```

FIX REVIEW NOTES

Fixed according to the recommendation: **calculate_fill** returns **u128**, the amounts are narrowed to **u64** only at the SPL transfers, and the partial-fill product goes through a 256-bit **muldiv_u128**.

L01: Front-run of **initialize** grants the first caller admin and the **portal_authority** escrow-release gate



LOCATION

liquidity-delivery/6bec28cb/svm/programs/order_book/src/instructions/initialize.rs:L8-L20 [↗](#)

RELEVANT SNIPPET

```
>_ initialize.rs RUST
6: use anchor_lang::prelude::*;
7:
8: #[derive(Accounts)]
9: pub struct Initialize<'info> {
10:     #[account(mut)]
11:     pub admin: Signer<'info>,
12:
13:     #[account(
14:         init,
15:         payer = admin,
16:         space = ANCHOR_DISCRIMINATOR_SIZE + OrderBookGlobal::INIT_SPACE,
17:         seeds = [GLOBAL_SEED],
18:         bump
19:     )]
20:     pub global_account: Account<'info, OrderBookGlobal>,
21:
22:     pub system_program: Program<'info, System>,
```

OTHER LOCATIONS

- liquidity-delivery/6bec28cb/svm/runbooks/deployment/main.tx [↗](#)
- liquidity-delivery/6bec28cb/svm/runbooks/initialize/main.tx [↗](#)

DESCRIPTION

initialize does not bind the caller to a known authority: **admin** is any **Signer**, and the **init** constraint on **[GLOBAL_SEED]** accepts whoever calls first. The instruction stores both **admin** and **portal_authority** from the caller's arguments:

RUST

```
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub admin: Signer<'info>, // any signer; no co-signer or deployer constraint

    #[account(init, payer = admin, space = ..., seeds = [GLOBAL_SEED], bump)]
    pub global_account: Account<'info, OrderBookGlobal>,
    ...
}
```

The program deploys and initializes in separate transactions: **deployment/main.tx** runs **svm::deploy_program**, and **initialize/main.tx** runs **initialize_orderbook** later. Anyone who calls **initialize** in between becomes **admin** and sets **portal_authority** to their own key. **portal_authority** is the only signer that can authorize **report_order_fill** and **report_order_cancel**, the instructions that release cross-chain escrow, so the winner can later authorize the release of any cross-chain order's escrow to any account they name. Recovery requires redeploying at a new program ID, because the global PDA persists across upgrades.

RECOMMENDATION

Require a deployer co-signer on **initialize**, or deploy and initialize atomically through a deployer program:

DIFF

```
#[derive(Accounts)]
pub struct Initialize<'info> {
+   #[account(address = DEPLOYER_PUBKEY)]
+   pub deployer: Signer<'info>,
+
    #[account(mut)]
    pub admin: Signer<'info>,
```

DEVELOPER RESPONSE

Acknowledged. The developers informed us that the program is already deployed and initialized, and that for a new deployment they would verify the config after **initialize** and close and redeploy the program if needed.

5. Enhancement Opportunities

E01: `ReportOrderFill::validate` does not check the reported `token_in` or reject a zero recipient

LOCATION

liquidity-delivery/6bec28cb/svm/programs/order_book/src/instructions/fill.rs:L587-L613 [↗](#)

RELEVANT SNIPPET

```

>_ fill.rs RUST

585:
586: impl ReportOrderFill<'> {
587:     fn validate(&self, fill_report: &FillReport) -> Result<()> {
588:         // Validate the order type is native
589:         require!(
590:             self.order.order_type == OrderType::Native,
591:             OrderBookError::InvalidOrderType
592:         );
593:
594:         let status = &self.order.data.status;
595:
596:         // Validate the order can be filled.
597:         // Note: Fill reports are allowed on Cancelled orders because cross-chain mess
        ages
598:         // may arrive out of order. A fill that occurred before cancellation on the de
        stination
599:         // chain might arrive after the cancel report on the origin chain. The amount
        tracking
600:         // (amount_in_released + amount_in_refunded <= amount_in) prevents over-distri
        bution.
601:         require!(
602:             status == &OrderStatus::Created || status == &OrderStatus::Cancelled,
603:             OrderBookError::OrderNotFillable
604:         );
605:
606:         // Validate the fill amount is not zero
607:         require!(
608:             fill_report.amount_out_filled > 0,
609:             OrderBookError::InvalidFillAmount
610:         );
611:
612:         Ok(())
613:     }
614:
615:     #[access_control(ctx.accounts.validate(&fill_report))]

```

DESCRIPTION

`ReportOrderFill::validate` checks the order type, the order status and `amount_out_filled > 0`, but never compares `fill_report.token_in` against `order.data.token_in` and never rejects a zero `fill_report.origin_recipient`. `ReportOrderCancel::validate` makes both comparisons on the equivalent cancel report, and **E04: origin_recipient zero-check is present on FillForeignOrder but absent on FillNativeOrder** covers `origin_recipient` on the separate `FillNativeOrder` path.

POTENTIAL BENEFIT

Aligns the fill report handler with **ReportOrderCancel::validate** and with the EVM implementation, both of which reject a **token_in** mismatch.

RECOMMENDATION

Mirror both checks in **ReportOrderFill::validate**.

DIFF

```
+   require_keys_eq!(  
+       Pubkey::new_from_array(fill_report.token_in),  
+       self.order.data.token_in,  
+       OrderBookError::InvalidTokenMint  
+   );  
+   require!(  
+       fill_report.origin_recipient != [0u8; 32],  
+       OrderBookError::InvalidRecipient  
+   );
```

DEVELOPER RESPONSE

Acknowledged.

E02: `set_portal_authority` rotates the escrow-release authority without staging or an event

LOCATION

liquidity-delivery/6bec28cb/svm/programs/order_book/src/instructions/admin.rs:L22-L32 [↗](#)

RELEVANT SNIPPET

```
>_ admin.rs RUST

20: impl AdminInstruction<'_> {
21:
22:     pub fn set_portal_authority(
23:         ctx: Context<Self>,
24:         new_portal_authority: Pubkey,
25:     ) -> Result<()> {
26:         require!(
27:             new_portal_authority != Pubkey::default(),
28:             OrderBookError::InvalidPortalAuthority
29:         );
30:         ctx.accounts.global_account.portal_authority = new_portal_authority;
31:         Ok(())
32:     }
33:
34:     pub fn set_new_admin(
```

DESCRIPTION

`set_portal_authority` overwrites `global_account.portal_authority` in a single instruction and emits no event. `portal_authority` is the required signer on `report_order_fill` and `report_order_cancel`, the only instructions that release cross-chain escrow, so reports already signed by the previous key fail with `NotAuthorized` and no indexer can observe the rotation. The `admin` role in the same file is staged through `set_new_admin` and `AcceptAdminRole`, and `AddDestination` emits `DestinationSupportUpdated`.

POTENTIAL BENEFIT

Makes the rotation observable off-chain, and prevents a key that cannot sign from taking over the escrow-release gate.

RECOMMENDATION

Emit an event on rotation, and stage the handover the way `admin` is staged.

```
DIFF

+ let old_portal_authority = ctx.accounts.global_account.portal_authority;
+ ctx.accounts.global_account.portal_authority = new_portal_authority;
+
+ emit_cpi!(PortalAuthorityUpdated {
+     old_portal_authority,
+     new_portal_authority,
+ });
```

DEVELOPER RESPONSE

Acknowledged.

E03: Writing **Nonce.bump** on initialization will complete the account's state

LOCATION

liquidity-delivery/6bec28cb/svm/programs/order_book/src/state/mod.rs:L9-L14 [↗](#)

RELEVANT SNIPPET

```
>_mod.rs RUST  
  
7: pub const NONCE_SEED_PREFIX: &[u8] = b"nonce";  
8:  
9: #[account]  
10: #[derive(InitSpace)]  
11: pub struct Nonce {  
12:     pub bump: u8,  
13:     pub value: u64,  
14: }  
15:  
16: #[constant]
```

DESCRIPTION

Nonce declares **bump: u8** and space is allocated for it, but no code path ever assigns it.

OpenOrder::handler only does **sender_nonce_account.value += 1**, so **bump** stays 0 on every nonce account.

Every other PDA in the program persists its bump on creation and validates against it afterwards: **OrderBookGlobal**, **Destination**, or **Order**.

POTENTIAL BENEFIT

Eliminates the one place where the program's PDA convention is broken.

RECOMMENDATION

```
DIFF  
  
+ if ctx.accounts.sender_nonce_account.bump == 0 {  
+     ctx.accounts.sender_nonce_account.bump = ctx.bumps.sender_nonce_account;  
+ }  
+  
// Increment the sender's nonce account  
ctx.accounts.sender_nonce_account.value += 1;
```

DEVELOPER RESPONSE

Acknowledged.

E04: **origin_recipient** zero-check is present on **FillForeignOrder** but absent on **FillNativeOrder**

LOCATION

liquidity-delivery/6bec28cb/svm/programs/order_book/src/instructions/fill.rs:L162-L184 [↗](#)

RELEVANT SNIPPET

```
> _fill.rs RUST

160:     )]
161:     pub order_token_in_ata: InterfaceAccount<'info, TokenAccount>,
162:
163:     pub token_in_program: Interface<'info, TokenInterface>,
164: }
165:
166: impl FillNativeOrder<'_> {
167:     fn validate(
168:         &self,
169:         order_id: &[u8; 32],
170:         order_data: &OrderData,
171:         fill_params: &FillParams,
172:     ) -> Result<()> {
173:         // Validate the params
174:         validate_params(order_id, order_data, fill_params, &self.solver.key())?;
175:
176:         // Validate the origin chain ID is this chain
177:         require!(
178:             order_data.origin_chain_id == self.global_account.chain_id,
179:             OrderBookError::InvalidOriginChainId
180:         );
181:
182:         // Validate the order is in a fillable state
183:         require!(
184:             self.order.data.status == OrderStatus::Created,
185:             OrderBookError::OrderNotFillable
186:         );
187:     }
188: }
```

DESCRIPTION

The two fill instructions treat **fill_params.origin_recipient** differently.

FillForeignOrder::validate rejects a zero value:

```
RUST

// Validate the origin recipient is not the zero address
require!(
    fill_params.origin_recipient != [0u8; 32],
    OrderBookError::InvalidRecipient
);
```

FillNativeOrder::validate has no such check. On the native branch, **origin_recipient** decides where the escrowed **token_in** goes: it is the required owner of **solver_token_in_account**.

POTENTIAL BENEFIT

Aligns **FillNativeOrder** with **FillForeignOrder**, which already validates the same field.

RECOMMENDATION

Mirror the foreign branch's check in **FillNativeOrder::validate**.

DEVELOPER RESPONSE

Acknowledged.

E05: Orders cannot be filled until the client's **created_at** guess expires

LOCATION

liquidity-delivery/6bec28cb/svm/programs/order_book/src/instructions/open.rs:L164-L174 [↗](#)

RELEVANT SNIPPET

```
>_open.rs RUST

162: );
163:
164: // On SVM, we allow the user to specify the created at timestamp to be within a small
165: // so that the PDA address can be precomputed off-chain without having to guess the ex
166: // act slot it will be included in.
167: let current_timestamp = Clock::get()?.unix_timestamp as u64;
168: require!(
169:     params.created_at >= current_timestamp,
170:     OrderBookError::InvalidCreatedAtTimestamp
171: );
172: require!(
173:     params.created_at <= current_timestamp + CREATED_AT_WINDOW,
174:     OrderBookError::InvalidCreatedAtTimestamp
175: );
176: // The fill deadline must be after the created at timestamp
177: require!(
```

OTHER LOCATIONS

- liquidity-delivery/6bec28cb/svm/scripts/open-order.ts:L203 [↗](#)

DESCRIPTION

open_order lets the client choose **created_at** so the order PDA can be precomputed off-chain. The code comment states this exists so the client does not have to "guess the exact slot it will be included in."

The validation window contradicts that goal. It requires:

```
RUST

params.created_at >= current_timestamp
params.created_at <= current_timestamp + CREATED_AT_WINDOW // now + 300s
```

So **created_at** must be in the future. This forces the client to guess, just in the other direction:

1. A client that sets **created_at = now** (the natural choice) reverts with **InvalidCreatedAtTimestamp** whenever its transaction lands even one second after signing.
2. The client compensates by guessing **created_at = now + 30**.

A backward-opening window (**now - 300 <= created_at <= now**) would fit the code comment as stated: the client can use the real creation time, the transaction is valid for the whole 300-second window, and the order is fillable and cancellable the moment it lands.

POTENTIAL BENEFIT

Orders become fillable as soon as they land, and the client does not have to guess a delay.

RECOMMENDATION

DIFF

```
let current_timestamp = Clock::get()?.unix_timestamp as u64;
    require!(
-         params.created_at >= current_timestamp,
+         params.created_at >= current_timestamp.saturating_sub(CREATED_AT_WINDOW),
        OrderBookError::InvalidCreatedAtTimestamp
    );
    require!(
-         params.created_at <= current_timestamp + CREATED_AT_WINDOW,
+         params.created_at <= current_timestamp,
        OrderBookError::InvalidCreatedAtTimestamp
    );
```

Update `open-order.ts` to use `createdAt = now` with no buffer.

DEVELOPER RESPONSE

Acknowledged. The developers informed us that their solvers and other solvers wait for `created_at` to pass before filling.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in Web3 audits.

The firm was built by a mix of experienced professionals from traditional enterprise security and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members whose publications have earned over 1,100 academic citations, and with the trust of projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the Web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is designed to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Finally, we examine external integrations with other Solana projects and verify that inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited, we may also include a detailed proof-of-concept exploit with code snippets and instructions on how the attack could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, express or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time of the engagement.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.